Trabalho Prático 2 - Sistema de Escalonamento Hospitalar

João Gabriel Guimarães Alves Vaz

Departamento de Ciência da Computação da UFMG (DCC/UFMG)
Belo Horizonte, Minas Gerais - Brasil
Matrícula: 2023099166
jgvaz@ufmg.br

2024

1 Introdução

O trabalho proposto tratava de um problema de simulação discreta de eventos em um hospital. Foi entregue um arquivo contendo informações sobre os pacientes (id, tempo de chegada, urgência no atendimento e número de exames a serem realizados) e sobre o número de atendentes disponíveis para atender aos pacientes. O objetivo do trabalho era simular as filas do hospital de forma a, ao final, imprimir estatísticas a respeito dos pacientes (i.e., tempo de saída e quanto desse tempo foi em atendimento e quanto foi em fila). A situação-problema deixa evidente a necessidade da implementação de um simulador de eventos (no caso, um simulador discreto de eventos, como foi descrito no enunciado) de forma a manter registro dos tempos e estados de cada paciente e das filas de cada procedimento.

Abaixo está um resumo do que será abordado em cada uma das seções desse documento.

- Na seção 1, há uma introdução ao tema, o contexto e o sumário.
- Na seção 2, há o método de implementação.
- Na seção 3, há a análise de complexidade dos procedimentos implementados.
- Na seção 4, há uma discussão das estratégias de robustez utilizadas.
- Na seção 5, há uma análise experimental.
- Na seção 6, há a conclusão desta documentação.
- Na seção 7, há a bibliografia.
- Na seção 8, há a heurística do ponto extra.

2 Método

O programa foi desenvolvido na linguagem C++, compilada pelo compilador G++ da GNU Compiler Collection

2.1 Tipos abstratos de dados

Para a modularização do código, foram implementados alguns tipos abstratos de dados. O principal é o TAD Paciente que, declarado como struct, armazena os dados dos pacientes: id, passível a alta antecipada, grau de urgência, momento de chegada, número de exames a serem realizados, momento de saída (tempo de alta) e tempo sendo atendido. Para a confecção da saída pedida, esses dados são manipulados (lidos como referência e modificados) ao longo do programa.

Além desse TAD, tem-se também o TAD Fila, Evento, Escalonador e Procedimento. Cada um desses guarda informações secundárias que auxiliam na execução do programa. Especificamente: Fila implementa uma fila padrão (com enfileira e desenfileira); Evento armazena um instante de tempo e um (ponteiro de) paciente associado; Escalonador é uma fila de prioridade (min-heap) usada para agendar e reagendar os eventos para garantir o fluxo correto do programa; e Procedimento contém informações (tempo de execução e número de guichês) dos procedimentos ao dispor do paciente (triagem, atendimento, medidas hospitalares, testes de laboratório, exames de imagem, instrumentos e medicamentos).

2.2 Módulos

Nesse trabalho, os módulos usados foram as implementações dos arquivos hpp de declarações de classes e métodos. Ou seja, as únicas funções existentes (além da main) são os métodos das classes implementadas. Nesse sentido, tem-se métodos getters e setters para variáveis do escalonador, dos eventos, das filas e dos procedimentos. O paciente, por ser um struct, não possui métodos.

2.3 Funcionamento do programa

O objetivo desta subseção é detalhar não as escolhas técnicas de implementação, mas sim o fluxo, em alto nível, do programa desenvolvido. Sendo um simulador de filas de hospital, a situação-problema apresentada requereu o uso de estruturas que simulassem o funcionamento de filas de pessoas na vida real. Sendo assim, fiz uso de estruturas auxiliares (além das filas em si) para abstrair a lógica temporal de prioridades da fila. Quem teve esse papel foi o escalonador (min-heap, ou fila de prioridade), que foi a estrutura responsável por definir qual paciente deveria ser processado em cada instante de tempo da simulação. Além disso, é essa fila de prioridade que garante a ordem dos eventos (ou seja, pacientes que chegaram antes ao hospital são processados antes de pacientes que chegaram depois). O escalonador armazena eventos (outra classe auxiliar), que é um wrapper para paciente e tempo. Ou seja, um evento possui um paciente e um instante de tempo associados, e o escalonador "ordena" esses eventos com base em seus tempos.

Detalhando com mais profundidade o fluxo lógico do programa:

- No início do programa, as variáveis de estado da simulação são definidas.
- Após isso, todos os pacientes são lidos da entrada e escalonados no escalonador.
- Já nessa fase inicial, é definido, para cada paciente, seu tempo em atendimento no hospital (já que essa grandeza é "fixa" - há uma fórmula fechada que a descreve, especificamente, a soma ponderada do número de exames multiplicado pela quantidade de exames que o paciente realizará).
- Nesse ponto, depois da leitura da entrada, começa a simulação propriamente dita. Na "primeira fase", os pacientes são, um por um, removidos do escalonador e processados quanto ao próximo exame que devem realizar ou se já finalizaram o processo e estão aptos a receberem alta.
- Ainda dentro do loop de repetição do escalonador: estando finalizado o processamento do paciente descrito acima, começa a "segunda fase". Nessa etapa, o paciente atual (que foi removido do escalonador) já foi processado, e portanto o objeto de processamento dessa fase são não mais os pacientes e sim as filas de atendimento. Uma por uma, as filas são "avançadas"; ou seja, verifica-se se há unidades de atendimento, "guichês", disponíveis e, caso positivo, as filas avançam, enviando os próximos pacientes para serem atendidos e reescalonando eventos no escalonador.
- E assim, finaliza-se o loop do escalonador. A lógica do programa garante que o escalonador só estará vazio quando todos os pacientes lidos da entrada tiverem recebido alta (pois somente quando um paciente recebe alta ele não é reescalonado).
- Então, a saída é impressa na tela (i.e., o tempo de permanência no hospital para cada paciente, mostrando quanto tempo foi passado em fila e quanto tempo foi passado em atendimento). Algo importante de ser explicado aqui é a forma como as informações de tempo foram armazenadas no programa. Tendo em vista que a saída requisita apenas o tempo de entrada e de saída de cada paciente (além da divisão tempo em fila/ tempo em atendimento), eu escolhi por armazenar apenas o tempo inicial (instante de chegada), final (instante de saída) e de atendimento (a soma ponderada mencionada acima) de cada paciente. Dessa forma, na impressão da saída, o tempo em

fila é calculado como: $fila = total - atendimento$, sendo que $total = final - inicial$. Assim, não foi necessário ter o tempo em fila armazenado explicitamente no paciente.

2.4 Instruções de compilação

Para executar o programa, basta chamar o utilitário *make* - que vai compilar o executável e colocá-lo em *./bin/tp2.out* - e chamar o programa, dessa forma:

```
make
./bin/tp2.out <entrada>
```

3 Análise de complexidade

3.1 Tempo

Como mencionado anteriormente, a função main é a responsável por chamar as demais funções. Portanto, em termos de análise assintótica, a complexidade da função é a complexidade do "módulo" (mais especificamente, do trecho de código) de maior complexidade dentre todos os trechos de códigos que compõem a função main. Realizando uma análise trecho a trecho:

- A leitura do arquivo de entrada percorre todas as n linhas do arquivo uma única vez, portanto, $O(n)$. Cada paciente é inserido no escalonador (implementado como um min-heap). A inserção de elementos num min-heap é, no pior caso, $O(log(n))$. Isso ocorre quando o elemento inserido é menor que todos os outros e tem que ser trocado com todos os demais até chegar na raiz para manter a propriedade do min-heap. Nessa ocasião, como o heap é uma árvore balanceada, são realizadas $log(n)$ trocas, fazendo o pior caso da inserção no heap ser $logn$. Como o laço inicial do programa faz a inserção no min-heap de todos os n pacientes e cada inserção tem um pior caso de $log(n)$, a complexidade do loop de leitura da entrada é $O(nlog(n))$.

- O loop do escalonador tira um paciente por vez, podendo ou não reescaloná-lo. Como um número constante de pacientes é inserido e removido por vez no escalonador, a complexidade temporal desse laço de repetição é ainda linear, $O(n)$. A remoção de um elemento em um min-heap tem complexidade $log(n)$, pelos mesmos motivos explicados acima. Como todos os n pacientes são (necessariamente) removidos do escalonador, sendo que alguns são reinseridos um número constante de vezes, a complexidade total do loop é um fator de $nlog(n)$. Dessa forma, a complexidade total do laço de repetição que executa enquanto o escalonador não está vazio é $O(nlog(n))$.

- As duas lambda-functions definidas são funções auxiliares que "fazem as filas andarem". $proximo()$ retira um paciente da fila e o escalona novamente, portanto, tem um conjunto fixo de instruções, sem laços de repetição que dependam do tamanho n da entrada. $avancar_fila()$ chama $proximo()$ enquanto houver unidades de atendimento ("guichês") disponíveis. O número de unidades de atendimento é fixo e não depende do tamanho da entrada. Assim, ambas têm complexidade temporal constante $O(1)$ e não alteram a complexidade do laço de repetição "pai" (o laço do escalonador).

- Por fim, há mais um laço linear de repetição, que percorre cada item da entrada (i.e., cada paciente) e imprime as informações no formato pedido no enunciado. Esse trecho também é linear, $O(n)$.

Logo, a complexidade temporal da função main é $O(nlog(n))$.

3.2 Espaço

A análise de complexidade de espaço é bem mais direta e simples que a temporal. Em termos de uso de memória, tem-se: 16 filas, 4 procedimentos, 1 min-heap e 1 vetor.

- As 16 filas foram implementadas de forma dinâmica. Ou seja, não há uma memória fixa pré-alocada para armazenar os itens da fila; a memória é alocada conforme necessidade. Ainda assim, pode-se argumentar de um "pior caso" no qual todos os n pacientes aparecem enfileirados em uma única fila. Tendo isso em vista, considera-se que o espaço ocupado pelas filas é no máximo linear (ou múltiplo de linear), $O(n)$.

- Os 4 procedimentos tomam espaço constante, pois servem apenas para armazenar o número de guichês/unidades de atendimento livres em cada instante de tempo, que também é constante (não depende do tamanho da entrada). Portanto, os 4 procedimentos tomam espaço $O(1)$.
- O min-heap é declarado com um tamanho fixo, n. Isso porque, da forma como o programa foi implementado, para cada paciente, um evento associado a ele pode ou não estar no min-heap; mas, caso esteja, há, em cada instante de tempo, apenas um evento associado a um paciente. Ou seja, no pior caso, há n eventos (em associação biunívoca aos n pacientes) no escalonador. Então, o escalonador é $O(n)$ em termos de complexidade de espaço.
- Por fim, o vetor de pacientes guarda um paciente por item. Logo, também é linear, $O(n)$ em armazenamento.

Dessa forma, em termos de complexidade espacial, o programa é $O(n + 1 + n + n) = O(3n) = O(n)$.

4 Estratégias de robustez

Ao longo do desenvolvimento deste trabalho, foi realizada a implementação de blocos de try-catch em diversas partes do código, como uma das principais estratégias de programação defensiva adotadas para garantir a robustez do sistema. Esses blocos desempenharam um papel fundamental ao permitir a detecção e o tratamento de exceções que poderiam ocorrer durante a execução do programa. A utilização dessa abordagem foi crucial para evitar falhas inesperadas que poderiam comprometer não apenas o funcionamento de componentes individuais, mas também a estabilidade geral da aplicação. Além de prevenir a interrupção abrupta da execução, os blocos de try-catch permitiram lidar com situações inesperadas de forma controlada, reduzindo os riscos associados a erros não tratados. Essa prática foi essencial para conferir maior confiabilidade ao sistema e para promover um desenvolvimento pautado na qualidade e na prevenção de problemas.

5 Análise experimental

Todos os testes abaixo detalhados foram feitos numa máquina com as especificações que se seguem. Processador: Intel i7, 7th gen, 8 cores. RAM: 16 GB. Armazenamento: 256GB SSD, 1TB HD. SO: Linux (Manjaro 24.0.4).

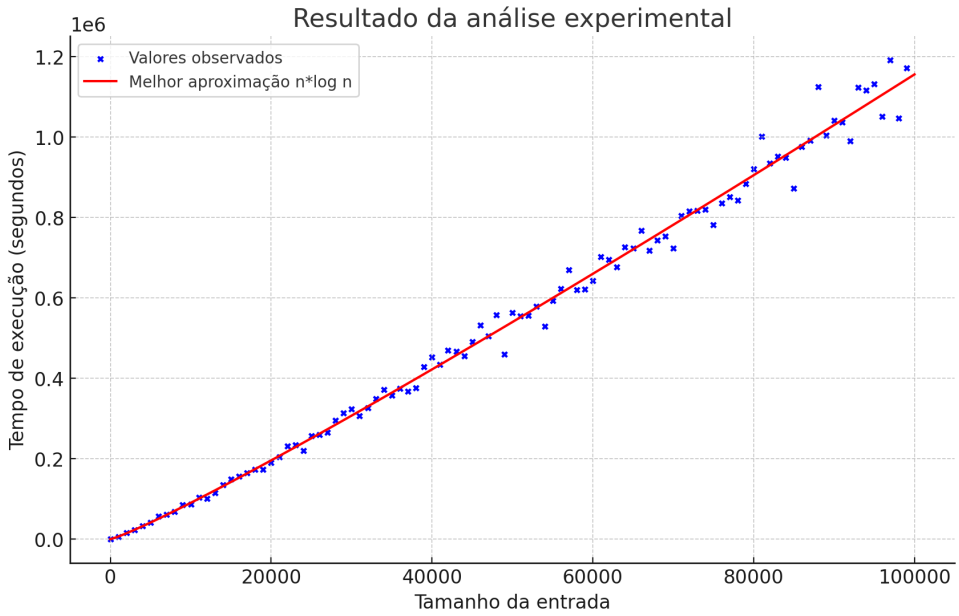


Figure 1: Os pontos azuis representam os "Valores observados" e a curva vermelha representa a "Melhor aproximação $n \log n$".

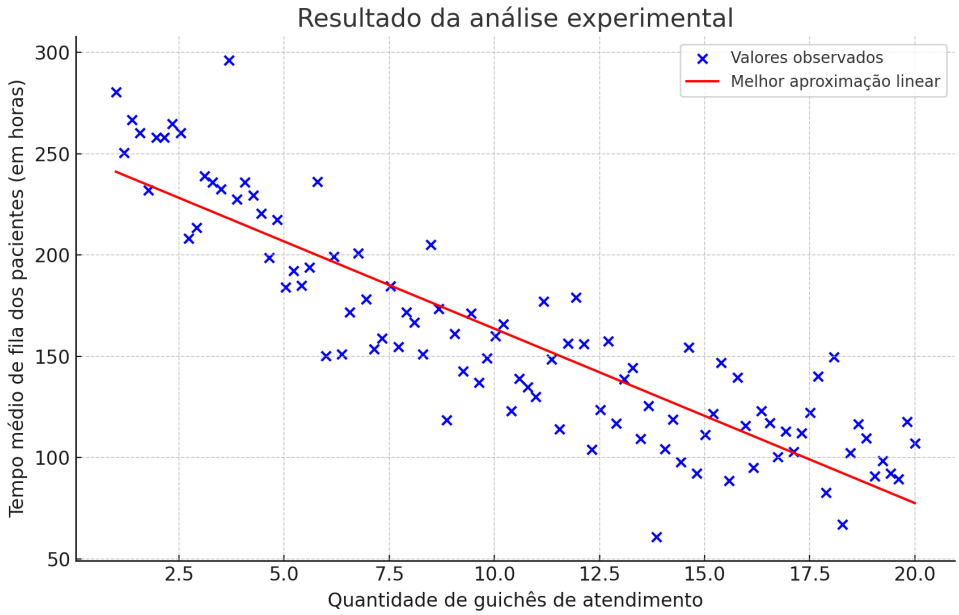


Figure 2: Os pontos azuis representam os "Valores observados" para o tempo médio de fila dos pacientes (em horas) conforme o número de guichês de atendimento varia de 1 a 20. A curva vermelha é a "Melhor aproximação linear" com um tamanho de entrada fixo de 10.000 pacientes.

A análise experimental revela que, de fato, a complexidade geral de tempo do programa se assemelha a uma $O(nlog(n))$. Isso faz sentido, tendo em vista que a complexidade temporal do programa é de fato $O(nlog(n))$, como descrito nas análises de complexidade.

Além disso, pode-se perceber, pelo segundo gráfico, que a relação entre a quantidade de guichês (unidades de atendimento) e tempo de fila aproxima-se de uma relação inversamente proporcional para um número n de pacientes fixo ($n = 10000$). Ou seja, o 2° gráfico mostra que é quase linearmente negativa a correlação entre a quantidade de guichês e o tempo de espera dos pacientes. Novamente, esse resultado também faz sentido, pois quanto mais guichês estiverem disponíveis para atender aos pacientes, menos tempo eles ficarão em fila.

6 Conclusões

Neste trabalho, foi possível aplicar conceitos fundamentais de estruturas de dados para desenvolver e simular um sistema complexo de filas e eventos discretos, representando o fluxo de atendimento hospitalar. Entre os principais aprendizados destacam-se:

- Modelagem e manipulação de dados: Implementação de Tipos Abstratos de Dados (TADs) para representar pacientes, procedimentos, filas e o escalonador, demonstrando a importância de encapsular comportamentos e dados de maneira eficiente.
- Estruturas dinâmicas: Uso de filas para gerenciar a espera e o atendimento, explorando operações como enfileirar, desenfileirar e calcular tempos de espera.
- Escalonamento por prioridade: Implementação de uma fila de prioridade com um min-heap para organizar eventos com base no tempo, destacando a importância de algoritmos eficientes.
- Simulação de eventos discretos: Criação de um sistema baseado em eventos, compreendendo como gerenciar estados, atualizar estatísticas e tratar múltiplos cenários em tempo simulado.
- Organização e análise de dados: Leitura de arquivos estruturados, manipulação de entradas e saídas e geração de estatísticas detalhadas para avaliar o desempenho e a eficiência do sistema.

O trabalho reforçou a aplicação prática de conceitos teóricos em cenários reais, combinando eficiência, organização e análise detalhada para solucionar problemas complexos. Alem disso, esse projeto abordou a implementação de um sistema de simulação para gerenciar o fluxo de pacientes em um hospital, representando de forma estruturada o processo desde a admissão até a alta hospitalar. A solução desenvolvida permitiu modelar diferentes etapas do atendimento, como triagem, procedimentos médicos e alta, utilizando estruturas de dados específicas, como filas e minheaps, para representar pacientes e procedimentos. A simulação de eventos discretos possibilitou organizar e escalonar atendimentos com base na prioridade e na disponibilidade de recursos, garantindo o tratamento adequado a diferentes graus de urgência. Além disso, o sistema gerou estatísticas detalhadas sobre o tempo de espera e atendimento, fornecendo subsídios para análises de desempenho e alocação eficiente de recursos hospitalares. Assim, o trabalho demonstrou a aplicação prática e eficiente de conceitos de estruturas de dados na resolução de problemas reais e complexos, relacionados à gestão de processos e otimização de sistemas.

7 Bibliografia

Meira, W. (2024). Slides virtuais da disciplina de estruturas de dados. Disponibilizado via moodle. Departamento de Ciência da Computação. Universidade Federal de Minas Gerais. Belo Horizonte.

8 Ponto extra

Uma adaptação pode ser feita no código de forma a reduzir o tempo em fila dos pacientes. Essa alteração, explicada diretamente no código por meio de comentários, consiste em colocar um paciente em alguma fila de exame se e somente se o paciente tiver exames daquela categoria a serem realizados. Ou seja, essa alteração permite que o paciente "pule" estados quando necessário (i.e., quando ele não possui exames de um determinado tipo a serem feitos). Além de pular estados, o paciente pula tempo gasto inutilmente em fila também. Da forma como o código foi implementado inicialmente, todos os pacientes entram em todas as filas independentemente se eles têm ou não exames de um certo tipo a serem realizados. Com a alteração proposta, o tempo de fila diminui graças à eliminação do "tempo extra" de fila pelo qual um paciente passa atualmente (esse tempo extra equivale ao tempo no qual ele fica na fila até ser atendido, não realizar exames, e sair da fila novamente para seguir com seus próximos procedimentos). Mostras da diminuição do tempo de fila podem ser vistas aqui e aqui - no qual a saída da esquerda é a saída com a heurística implementada e a da direita, sem. Pode-se perceber que o tempo de fila com a heurística é majoritariamente menor que o tempo sem ela.